

The Data Pipeline, Explained in Simple Terms

Read this if you want to understand what the data layer does without touching the code. No jargon. For run commands and API details, see `README.md`.

1. What are we building, in one line?

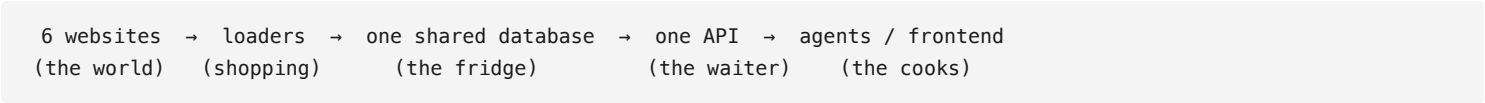
A system that **watches the world for danger to India's oil supply** and, the moment something looks bad (say, trouble near the Strait of Hormuz), instantly answers: *how bad is it, what does it cost India, and what should we buy instead?*

For that, the system needs **eyes** — real data flowing in continuously. This repo is those eyes. All of it is free, real, live data. Nothing is made up.

2. The 6 "eyes" (our data sources)

#	Source	What it tells us	Simple description
1	EIA (US govt)	Official oil price, daily	The trusted price record — Brent & WTI, every day since decades
2	yfinance (Yahoo)	Oil price, minute-by-minute	The "price is ticking up RIGHT NOW" signal
3	OFAC (US Treasury)	Sanctions list	~19,000 banned people/companies/ships, ~1,500 of them ships
4	AISStream	Live ship GPS	Actual positions of real ships at sea, streaming live
5	PPAC (Indian govt)	India's import bill	How much crude India buys monthly and what it pays — this is how we say "this crisis costs ₹X crore"
6	GDELT	World news	A free service that reads news from the whole planet; we keep only the scary events (strikes, sanctions, conflict near oil routes) and give each a severity score

3. How the data flows (think of a kitchen)



Step 1 — Shopping (the loaders). One small Python script per source. Its only job: go to that website, download fresh data, clean it, put it in the database. One folder per source under `datapipeline/`.

Step 2 — The fridge (Postgres on Neon). One shared cloud database. Everyone on the team — agents, models, frontend — reads from this *same fridge*. Nobody talks to the websites directly. That's why data formats never mismatch between teammates.

Step 3 — The cook who restocks (`controller.py`). One script that knows how often each ingredient goes stale: prices every ~10 minutes, sanctions daily, India's import data monthly, news every few hours. Run it and it refreshes only what's due. It also throws out exactly one thing: **ship positions older than 2 days** (a ship's Tuesday location is garbage by Thursday). Everything else is kept forever, because our maths model *learns from history* — deleting old prices would break it.

Step 4 — The robot that runs the cook (GitHub Actions). GitHub runs our controller every 30 minutes on its own computers, free. So the database stays fresh 24/7 even when every laptop is off. (Needs 3 secrets added in repo settings to switch on: `DATABASE_URL` , `EIA_API_KEY` , `AISSTREAM_API_KEY` .)

Step 5 — The waiter (the API, `api/main.py`). One FastAPI server in front of the database. Anyone asks simple HTTP questions — `/latest Brent price?` , `"all ships right now?"` , `"how risky is Hormuz today?"` — and gets clean JSON. Nobody needs SQL or the database password. Interactive docs at `/docs` .

4. The news pipeline (the 6th eye, in slightly more detail)

News is messy text, so it gets its own mini-assembly-line (`datapipeline/gdelt_and_gkg/`):

1. **Fetch** — download the last 30 days of world events. Two ways: the *public* way (free GDELT daily files, no account — the default) or the *BigQuery* way (richer data, needs a Google account).
2. **Merge** — attach related news articles to each event as context.
3. **Extract** — turn each raw event into one clean "risk signal": *who did what, where, how severe (0–10), how confident are we (0–1), which sea corridor does it affect*. Done by fast rules; an optional LLM reviews only the unclear ones.
4. **Load** — save the signals into the `structured_events` table.

From these signals the API computes a **risk score per corridor** (0 to 1): recent + severe + well-confirmed events push it up; old news fades away automatically (an event from 7 days ago counts half as much as today's).

5. The knowledge graph (the newest piece)

A **map of things and how they connect** — 32 real nodes, 48 real connections:

```
supplier country → its export port → sea chokepoints → Indian port → refinery
```

It knows real infrastructure, including the "secret exits": Saudi Arabia's pipeline to Yanbu on the Red Sea, UAE's pipeline to Fujairah *outside* Hormuz, Russia's pipeline to the Pacific. Iraq and Kuwait have none — a true fact the graph reproduces.

The clever bit: the graph is alive. The map itself is fixed (geography doesn't change), but the *danger* on each chokepoint updates from the real news events in our database. So asking `/graph/alternatives` literally answers: *"given TODAY's news, who can still deliver crude to Jamnagar, by which route, in how many days?"* — computed, not guessed by an LLM.

Real test result (with live data): Hormuz risk 0.89 and Red Sea 0.90 → the graph answered: UAE via Fujairah (3 days), Nigeria via the Cape (26.5 days), USA via the Cape (34.5 days), and correctly declared Iraq & Kuwait stuck. Nobody hardcoded that table — it fell out of the graph.

6. Two "wow" features already working

- **Live corridor risk:** Strait of Hormuz currently scores ~0.88 ("Critical") — computed from 200+ real news events, each one listable as evidence.
 - **Sanctioned-ship detection:** every live ship on the map is cross-checked against the US sanctions list — by MMSI number when the list provides it (strong match) or by name (weak match, labeled as such). Banned tankers get flagged the moment they appear.
-

7. Problems we hit and how we solved them (honest list)

Problem	Solution
News pipeline had hardcoded dates (only ever fetched June 1 – July 10)	Rolling window: always "last 30 days from today"
News pipeline needed one person's Google account	New public fetcher — anyone (and GitHub's robot) can run it with zero accounts
BigQuery queries were scanning terabytes (would burn the free quota in a few runs)	Switched to date-partitioned tables — ~100× less scanned
The free ship-tracker has no coverage near Hormuz (receivers are community-run, clustered in Europe)	Default box is the English Channel to prove the pipeline is live with real ships; the Hormuz story is told through geography + sanctions + news signal — all real data we do have
India's import data site blocks scraping	The monthly CSV is downloaded once and committed; swap the file to refresh
"Delete old data" idea would have broken the maths model	We prune only stale ship positions; price/news/import history is kept — the model calibrates on it

8. What's deliberately simple (and why that's fine)

- Severity/confidence formulas are sensible heuristics, not trained models — they're transparent and explainable on stage.
- Risk scores from very few events are statistically thin — so the API now **flags them** (`low_evidence: true` under 10 events, and the graph reports how many events back each chokepoint's risk).
- Voyage days and import shares in the graph are labeled approximations of real figures.
- Rule of the project: **real-but-cached beats fabricated. Never invent numbers.**

8b. Hardening added before shipping

- **Geofenced corridors:** every news event carries its real coordinates now; an event *at* the Strait of Hormuz and a protest in inland Iran are no longer lumped together. Coordinates are stored, so the map can plot the actual evidence dots.
- **Input validation:** crude prices outside \$1–500/bbl are rejected as data errors; a truncated sanctions download fails loudly instead of silently loading a partial list.
- **Retries:** downloads retry with backoff, so one network blip in the cloud refresher doesn't drop a feed for 30 minutes.
- **Tests + CI:** 25+ unit tests (extraction rules, graph routing, dump parser) run automatically on every push via GitHub Actions.
- **Staleness flags:** `/freshness` marks any feed that hasn't refreshed within ~2× its expected cadence, so the dashboard can show "data overdue" honestly.

9. Current status & what's next

Done: all 6 feeds live in the shared database · controller · unified API · knowledge graph · cloud auto-refresh workflow (waiting on the 3 repo secrets).

Next (not part of this repo yet): the LangGraph agents that read this data and produce the recommendation, the Monte-Carlo price-shock model, and the one-screen React + Leaflet dashboard with the 47-second timer.

The data layer is finished. Build on top of it; don't rebuild it.